《计算机图形学及人机交互》

实验报告

学　号：2023112621

姓　名：程泽凯

年　级：2023 级

专　业：计算机科学与技术

院　系：计算机与人工智能学院

2025 2026 第 1 学期

1 实验目的

通过本次实验，熟悉计算机图形学开发环境和工具，掌握 Direct2D 图形编程的基本方法，学会使用 Visual Studio 2022 进行图形应用程序的开发。具体目标包括：

1. 熟悉 Windows 图形编程环境
2. 掌握 Direct2D API 的基本使用方法
3. 学会绘制基本几何图形（直线、矩形、椭圆、圆弧等）
4. 理解面向对象的图形程序设计思想
5. 培养图形程序调试和优化能力

2 实验任务

参看下面的图形，请采用 GDI 或 Direct2D 编程实现。其中标注部分的内容可以不绘制。
要求：

1. 采用 Direct2D 实现（5 分）。
2. 撰写实验报告，主要包括以下几个部分的内容：（共 95 分）
   (a) 给出本次实验的目的，任务及要求等（5 分）。
   (b) 根据任务要求，给出程序的主要功能模块结构图（15 分），描述每个功能模块实现的主要功能（10 分）。
   (c) 给出每个模块的程序流程图（20 分）、完整的程序代码（30 分）及程序运行结果的屏幕截图（10 分）。
   (d) 实验总结（5 分）。

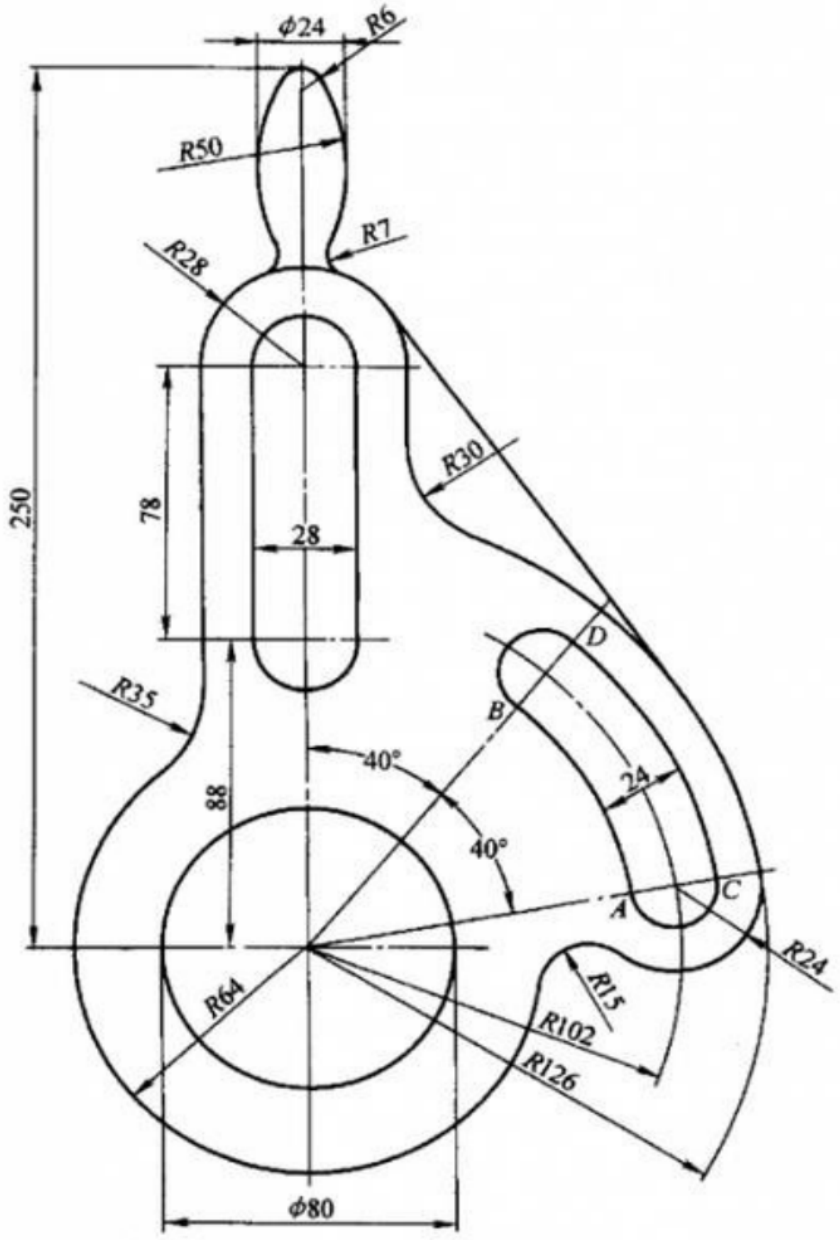


图 1: 实验要求绘制的图形

3 实验环境

3.1 硬件环境

1. 计算机：机械革命旷世 16pro
2. CPU：12th Gen Intel® i9-12900h
3. RAM：32G

3.2 软件环境

1. 操作系统：Windows 11 专业版
2. 开发工具：Visual Studio 2022

4 图形坐标计算

在实现复杂图形绘制之前，需要先进行精确的坐标计算。本节将详细分析图形中所有关键点的坐标计算过程和绘制顺序。

4.1 坐标系统设定

根据实验要求，建立以窗口左上角为原点的坐标系，向右为 X 轴正方向，向下为 Y 轴正方向。窗口尺寸设定为 400×500 像素。

坐标系参数：

- 窗口宽度：400 像素
- 窗口高度：500 像素
- 原点位置：窗口左上角 (0, 0)
- X 轴方向：向右为正
- Y 轴方向：向下为正

4.2 主要图形元素坐标计算

根据坐标计算图和绘制顺序表，对图形中的各个关键点进行精确定位：

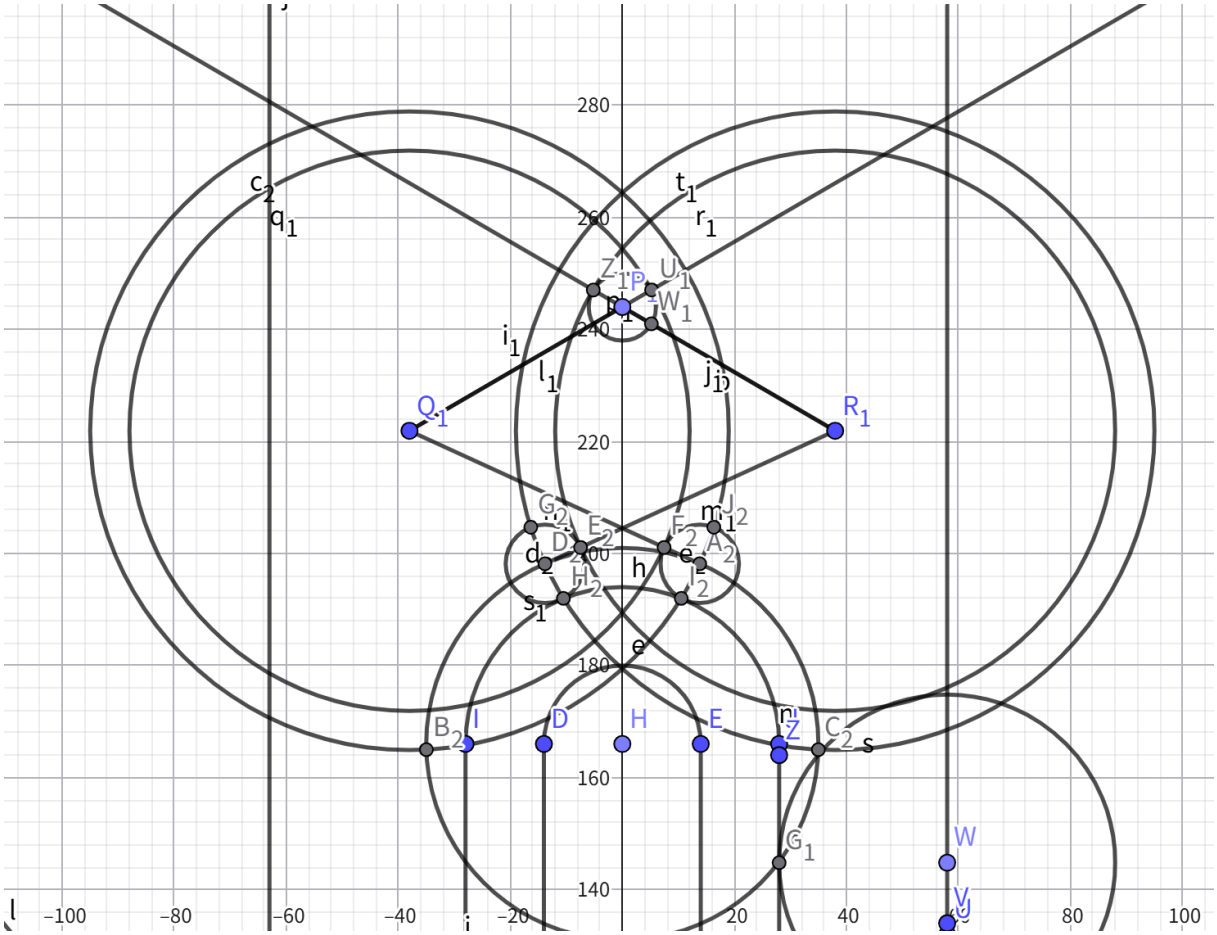


图 2: 图形坐标计算图（上半部分）

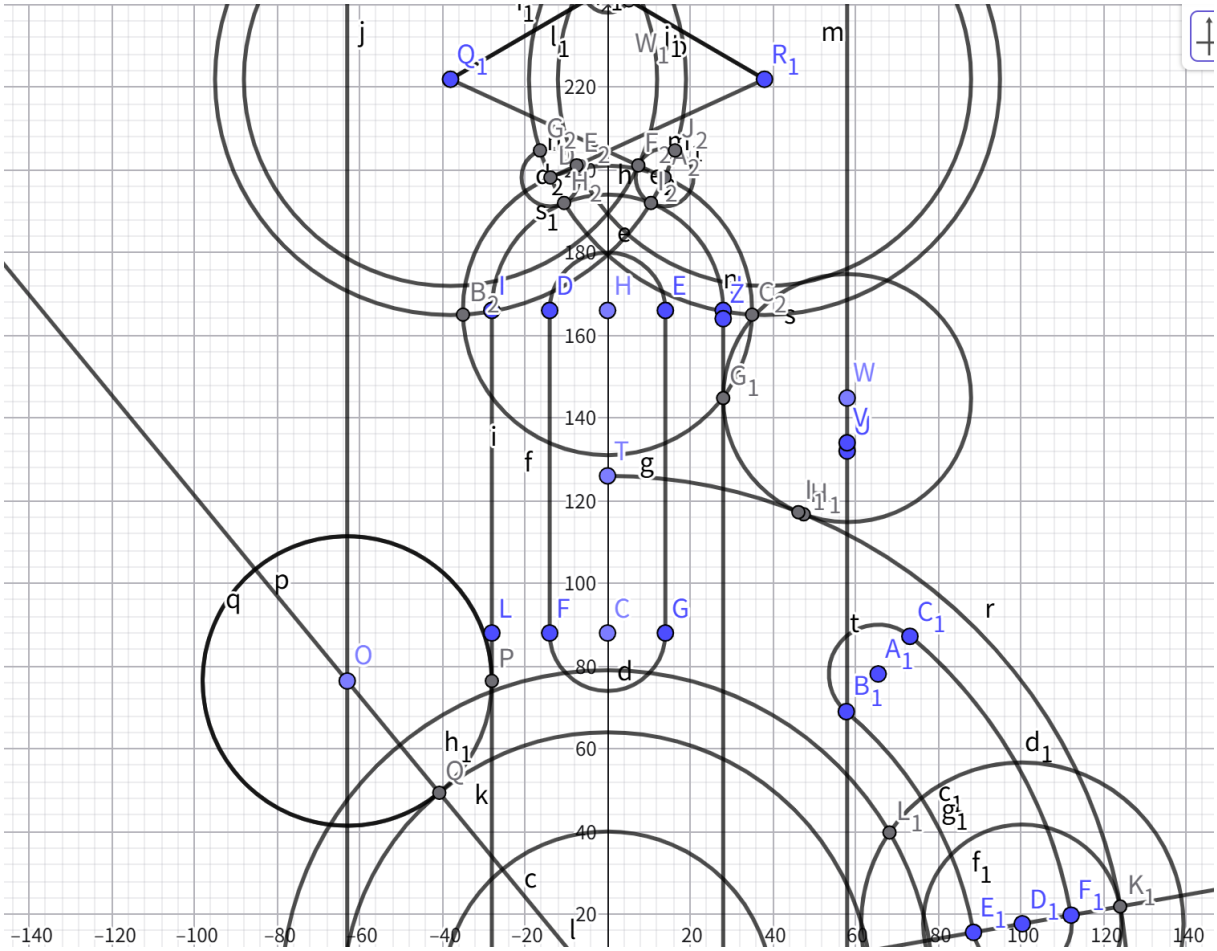


图 3: 图形坐标计算图（中间部分）

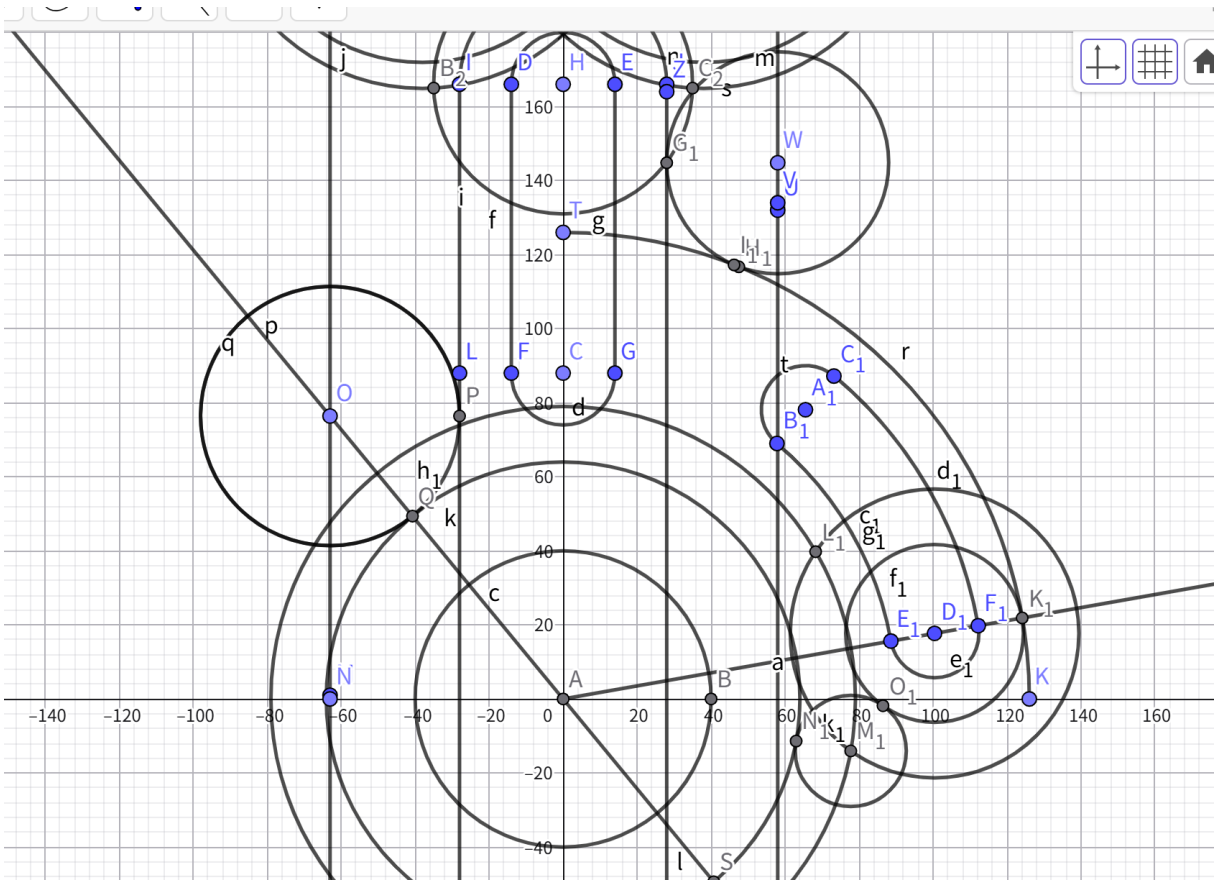


图 4: 图形坐标计算图（下半部分）

4.2.1 中心区域关键点

表 1: 中心区域关键点坐标

点名	X 坐标	Y 坐标	描述
A	150	300	中心椭圆心
B	122	160	上方垂直线起点
C	122	223.6	上方垂直线终点
D	87	223.6	左上圆弧起点
E	122	134	左上圆弧终点

4.2.2 外层圆弧关键点

表 2: 外层圆弧关键点坐标

点名	X 坐标	Y 坐标	描述
Q	87	223.6	左上大圆弧起点
R	213	223.6	右上大圆弧终点
F	0	80	左下圆弧中点
G	40	80	右下圆弧中点

4.2.3 辅助线段关键点

表 3: 辅助线段关键点坐标

点名	X 坐标	Y 坐标	描述
L	-40	80	左侧斜线起点
M	80	40	左侧斜线终点
N	-60	0	左下斜线起点
O	-100	80	左下斜线终点

4.3 圆弧参数计算

对于复杂的圆弧绘制，需要计算圆心、半径、起始角度和终止角度：

4.3.1 主要圆弧参数

表 4: 圆弧参数计算结果

圆弧名称	圆心 X	圆心 Y	半径	起始角	终止角
中心椭圆	150	300	40	0°	360°
左上圆弧	87	223.6	35	109.28°	250.62°
右上圆弧	213	223.6	35	289.38°	70.62°
大圆弧	150	300	64	212.98°	327.02°

4.4 角度计算方法

对于圆弧的角度计算，采用以下数学公式：

角度计算公式：

$$\theta = \arctan\left(\frac{y - cy}{x - cx}\right) \tag{1}$$

$$\text{角度标准化} : \theta \in [0, 2\pi] \tag{2}$$

$$\text{弧度转角度} : \text{degree} = \theta \times \frac{180}{\pi} \tag{3}$$

其中：

- (x, y) 为圆弧上的点坐标
- (cx, cy) 为圆心坐标
- θ 为计算得到的角度（弧度）

4.5 坐标变换与调整

在实际编程实现中，需要将理论计算的坐标转换为适合窗口显示的像素坐标。主要调整包括：

4.5.1 坐标平移变换

由于原始计算中存在负坐标，需要将整个图形向右下方平移，确保所有点都在窗口可视区域内。

坐标变换公式：

$$x_{\text{screen}} = x_{\text{original}} + \text{offset}_x \tag{4}$$

$$y_{\text{screen}} = y_{\text{original}} + \text{offset}_y \tag{5}$$

平移参数：

- $\text{offset}_x = 150$ （向右平移 150 像素）
- $\text{offset}_y = 50$ （向下平移 50 像素）

4.5.2 实际使用的坐标值

经过坐标变换后，主要关键点的实际坐标为：

表 5: 实际使用的关键点坐标

点名	理论坐标	实际 X	实际 Y	用途
A	(0, 250)	150	300	中心椭圆心
B	(-28, 110)	122	160	垂直线起点
C	(-28, 173.6)	122	223.6	垂直线终点
D	(-63, 173.6)	87	223.6	左上圆弧起点
E	(-28, 84)	122	134	左上圆弧终点

4.6 绘制顺序设计

为了确保图形的正确显示效果，制定了以下绘制顺序：

1	点 A	x轴 与 y轴 的交点	A = (0, 0)
2	圆 c	圆心为 A 且半径为 40 的圆	c: $x^2 + y^2 = 1600$
3	点 B	c 与 x轴 的交点	B = (40, 0)
4	点 C	y轴 上的点	C = (0, 88)
5	点 D		D = (-14, 166)
6	点 E		E = (14, 166)
7	弧线 e	经过 D 与 E 的半圆	e = 43.98
8	点 F		F = (-14, 88)
9	点 G		G = (14, 88)
10	弧线 d	圆弧(C, F, G)	d = 43.98
11	线段 f	线段DF	f = 78
12	线段 g	线段EG	g = 78
13	点 H	y轴 上的点	H = (0, 166)
14	点 J		J = (28, 166)
15	圆 k	圆心为 A 且半径为 64 的圆	k: $x^2 + y^2 = 4096$
16	点 I		I = (-28, 166)
17	弧线 h	圆弧(H, J, I)	h = 87.96
18	点 L		L = (-28, 88)
19	射线 i	端点为 I 且经过点 L 的射线	i: x = -28
20	点 M		M = (-63, 1)
21	点 N	x轴 上的点	N = (-63, 0)
22	直线 j	直线 MN	j: x = -63
23	点 O	j 上的点	O = (-63, 76.4)
24	圆 p	圆心为 O 且半径为 35 的圆	p: $(x + 63)^2 + (y - 76.4)^2 = 1225$
25	点 P	p 与 i 的交点	P = (-28, 76.4)
26	点 R	p 与 k 的交点	R = (?, ?)
27	直线 l	直线 OA	l: 76.4x + 63y = 0
28	点 Q	k 与 l 的交点	Q = (-40.72, 49.38)
28	点 S	k 与 l 的交点	S = (40.72, -49.38)
29	弧线 q	圆弧(O, P, Q)	q = 189.07
30	点 K	x轴 上的点	K = (126, 0)
31	点 T	y轴 上的点	T = (0, 126)
32	弧线 r	圆弧(A, K, T)	r = 197.92
33	点 U		U = (58, 132)
34	点 V		V = (58, 134)
35	直线 m	直线 UV	m: x = 58
36	点 W	m 上的点	W = (58, 144.81)
37	圆 s	圆心为 W 且半径为 30 的圆	s: $(x - 58)^2 + (y - 144.81)^2 = 900$
38	点 Z		Z = (28, 164)
39	射线 n	端点为 J 且经过点 Z 的射线	n: x = 28
40	点 A_1(65.5, 78.1)		A_1 = (65.5, 78.1)
41	点 B_1(57.8, 69)		B_1 = (57.8, 69)
42	点 C_1(73.2, 87.2)		C_1 = (73.2, 87.2)
43	弧线 t	圆弧(A_1, C_1, B_1)	t = 37.45
44	点 D_1(100.4, 17.7)		D_1 = (100.4, 17.7)
45	点 E_1(88.6, 15.62)		E_1 = (88.6, 15.62)
46	弧线 c_1	圆弧(A, E_1, B_1)	c_1 = 62.89
47	点 F_1(112.2, 19.78)		F_1 = (112.2, 19.78)
48	弧线 d_1	圆弧(A, F_1, C_1)	d_1 = 79.52
49	弧线 e_1	圆弧(D_1, E_1, F_1)	e_1 = 37.64
50	点 G_1(28, 144.81)	s 与 n 的交点	G_1 = (28, 144.81)
51	点 H_1(47.5, 116.7)	s 与 r 的交点	H_1 = (47.5, 116.7)
51	点 I_1(46.19, 117.23)	s 与 r 的交点	I_1 = (46.19, 117.23)
52	圆 f_1	圆心为 D_1 且半径为 24 的圆	f_1: $(x - 100.4)^2 + (y - 17.7)^2 = 576$
53	点 J_1(?, ?)	f_1 与 r 的交点	J_1 = (?, ?)
54	射线 a	端点为 A 且经过点 D_1 的射线	a: -17.7x + 100.4y = 0
55	点 K_1(124.09, 21.88)	r 与 a 的交点	K_1 = (124.09, 21.88)
56	圆 g_1	圆心为 D_1 且半径为 39 的圆	g_1: $(x - 100.4)^2 + (y - 17.7)^2 = 1521$
57	圆 h_1	圆心为 A 且半径为 79 的圆	h_1: $x^2 + y^2 = 6241$
58	点 L_1(68.25, 39.78)	h_1 与 g_1 的交点	L_1 = (68.25, 39.78)
58	点 M_1(77.74, -14.04)	h_1 与 g_1 的交点	M_1 = (77.74, -14.04)
59	圆 k_1	圆心为 M_1 且半径为 15 的圆	k_1: $(x - 77.74)^2 + (y + 14.04)^2 = 225$
60	点 N_1(62.98, -11.38)	k 与 k_1 的交点	N_1 = (62.98, -11.38)
61	点 O_1(86.46, -1.83)	f_1 与 k_1 的交点	O_1 = (86.46, -1.83)
62	点 P_1(0, 244)	y轴 上的点	P_1 = (0, 244)
63	圆 p_1	圆心为 P_1 且半径为 6 的圆	p_1: $x^2 + (y - 244)^2 = 36$
64	点 Q_1(-38, 221.9)		Q_1 = (-38, 221.9)
65	点 R_1(38, 221.9)		R_1 = (38, 221.9)
66	圆 q_1	圆心为 Q_1 且半径为 50 的圆	q_1: $(x + 38)^2 + (y - 221.9)^2 = 2500$
67	圆 r_1	圆心为 R_1 且半径为 50 的圆	r_1: $(x - 38)^2 + (y - 221.9)^2 = 2500$

1. 中心圆：作为主体结构，首先绘制
2. 外层大圆弧：绘制包围的大圆弧结构
3. 左上圆弧：绘制左上角的小圆弧
4. 右上圆弧：绘制右上角的小圆弧
5. 垂直线段：连接上下结构的线段
6. 辅助线段：其他装饰性线段

绘制顺序的设计原则：

- 从主体到细节：先绘制主要结构，再添加细节
- 从内到外：先绘制内部结构，再绘制外层结构
- 避免遮挡：确保后绘制的元素不会遮挡先绘制的元素

5 实验步骤与结果

5.1 绘制程序的主要功能模块结构图

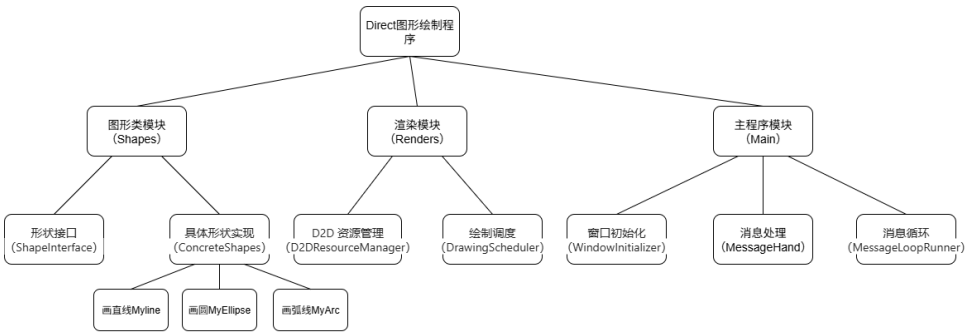


图 6: 程序模块树状结构图

5.2 程序模块简要介绍

根据程序模块树状结构图，本程序采用层次化模块设计，共包含 4 个父模块和 9 个子模块。以下是各模块的功能简介：

5.2.1 系统依赖模块

模块功能：负责引入程序所需的所有外部依赖库和头文件。
主要组件：

- Windows API 基础库（windows.h）
- Direct2D 图形库（d2d1.h）
- STL 容器库（vector）
- 数学函数库（cmath）
- 内存管理库（new）

作用：为其他模块提供基础的系统功能支持，确保程序能够正常调用 Windows API 和 Direct2D 图形接口。

5.2.2 图形类父模块（Shapes）

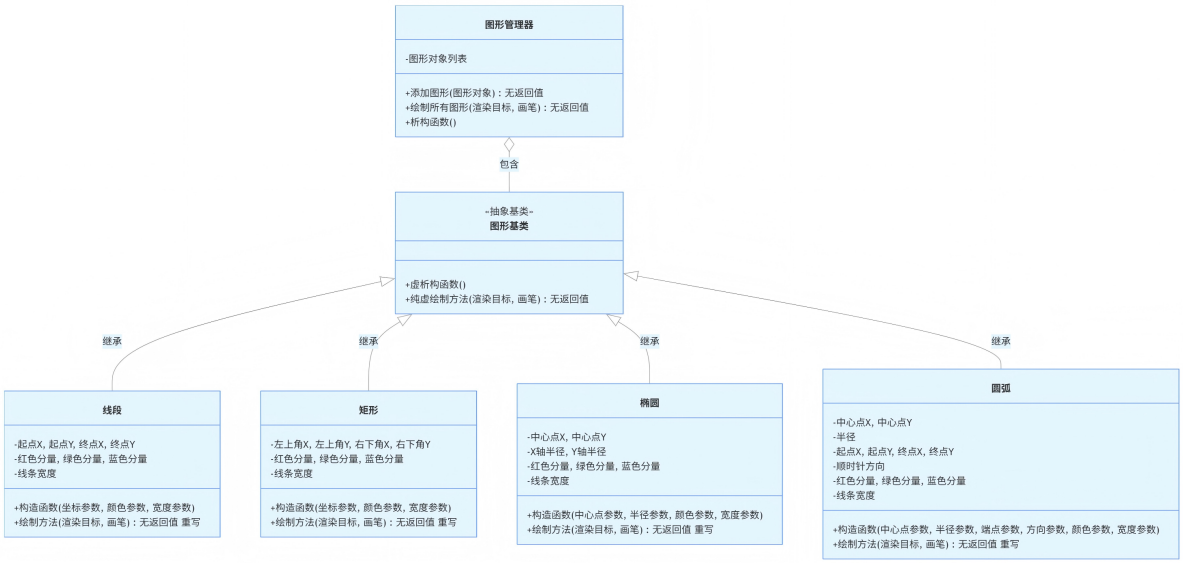


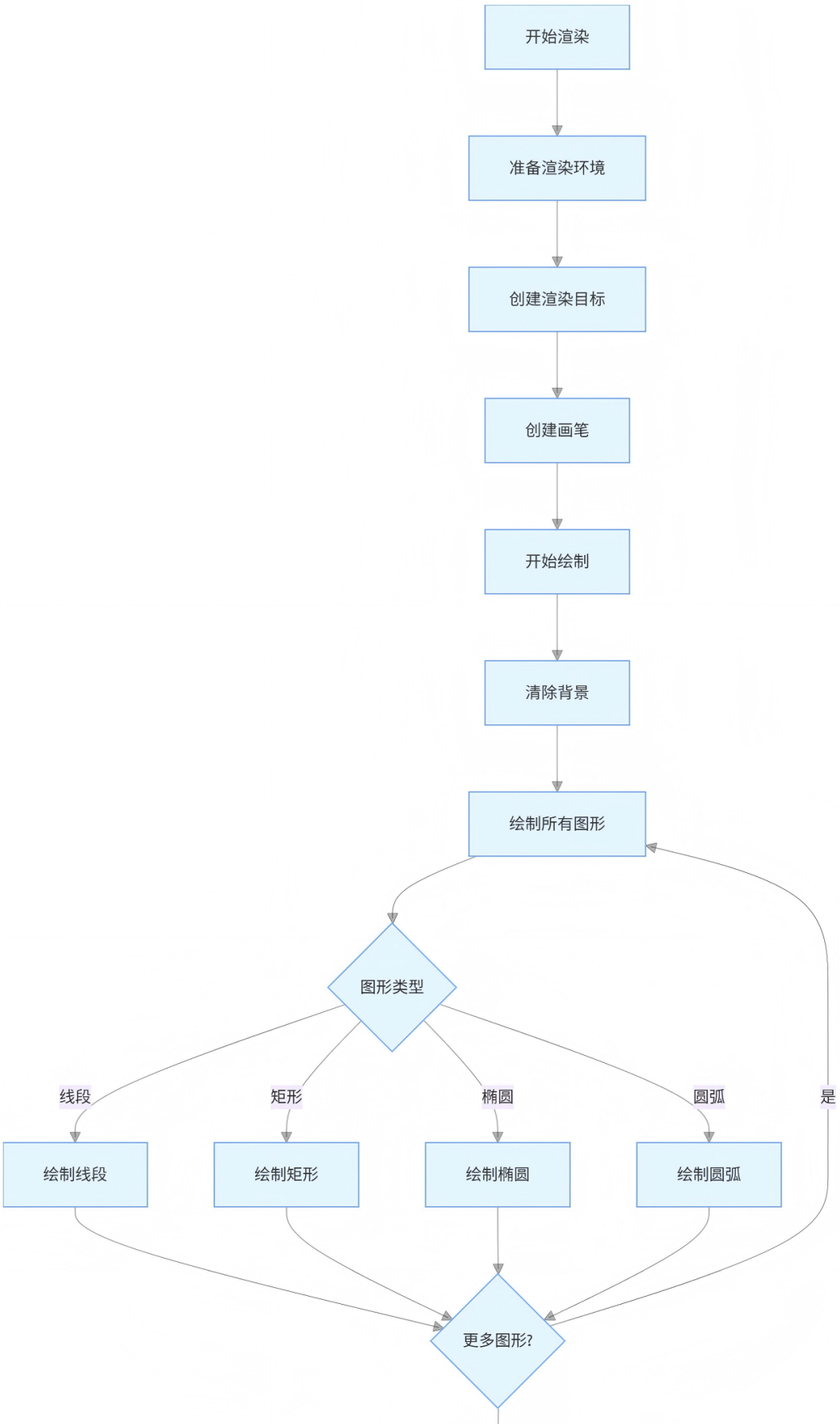
图 7: 图形类模块流程图）

模块功能：定义和管理所有图形对象，提供统一的图形绘制接口。
子模块组成：

- 形状接口子模块：定义 Shape 基类，提供统一的绘制接口
- 具体形状实现子模块：实现直线、矩形、椭圆、圆弧四种具体图形类

设计特点：采用面向对象设计，通过继承和多态实现图形的统一管理和绘制。

5.2.3 渲染父模块（Renderer）



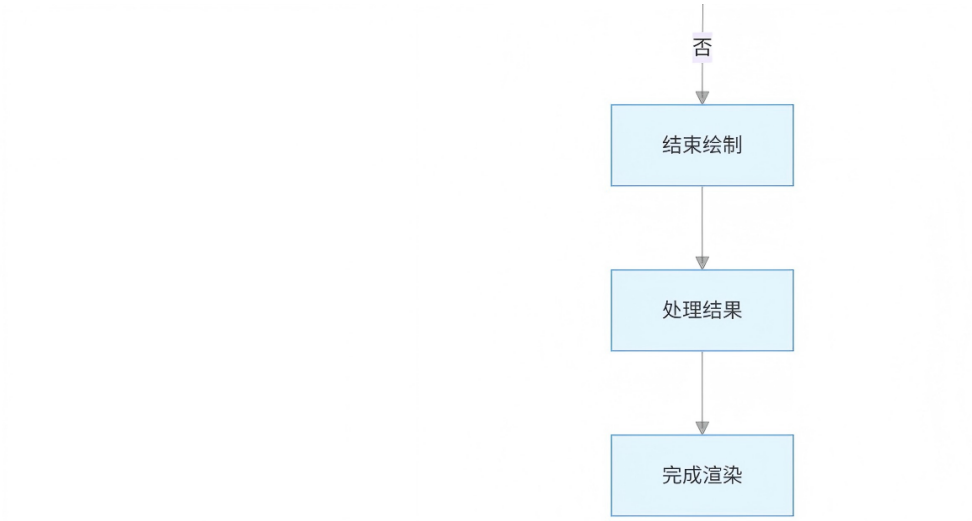


图 8: 渲染模块流程图

模块功能：管理整个图形渲染流程，负责 Direct2D 资源管理和绘制调度。
子模块组成：

- **D2D** 资源管理子模块：管理 Direct2D 工厂、渲染目标、画刷等资源
- 绘制调度子模块：维护图形对象列表，执行批量绘制操作

核心功能：协调图形对象的创建、存储和渲染输出。

5.2.4 主程序父模块（Main）

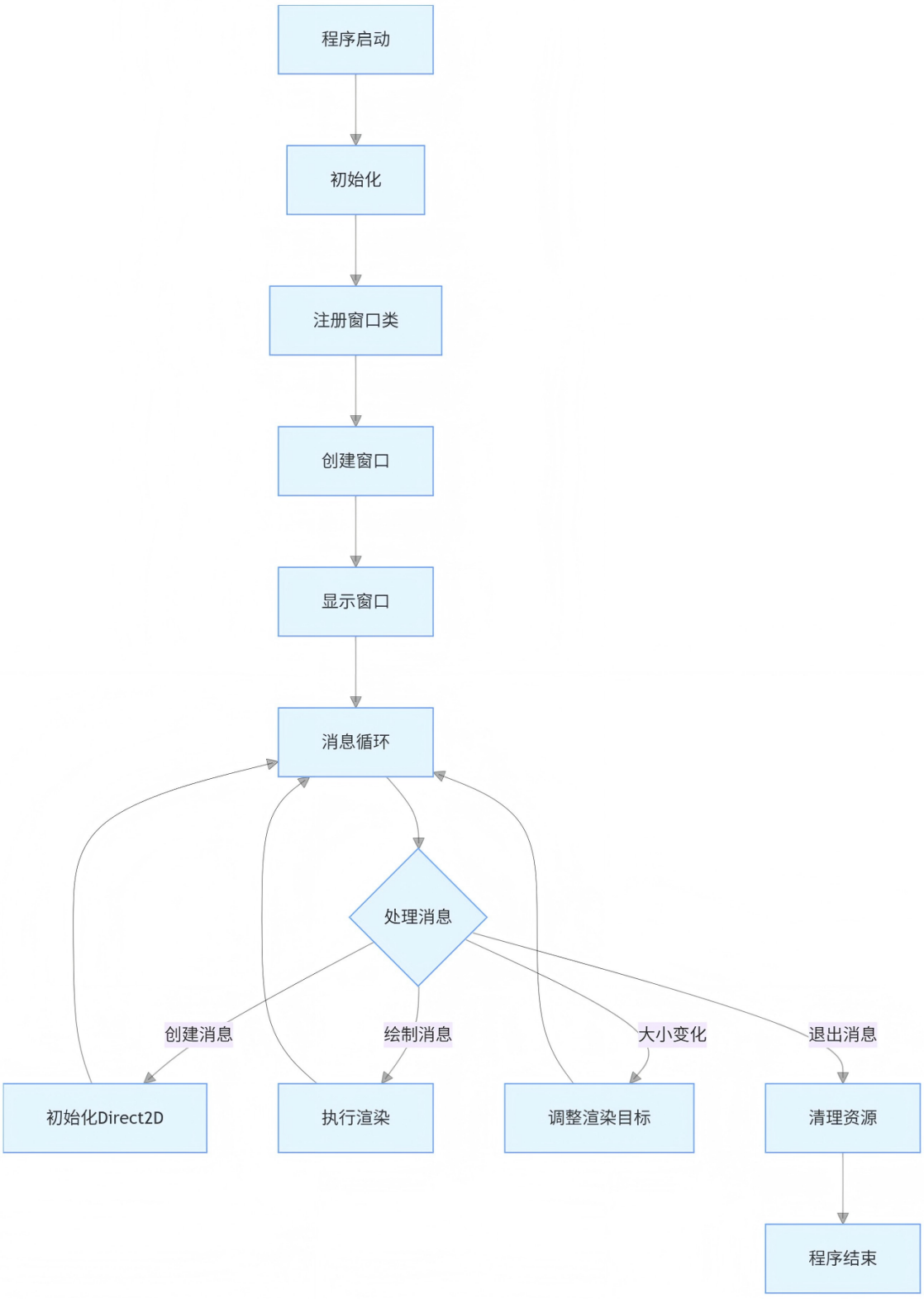


图 9: 主程序模块流程图

模块功能：程序的最高层控制模块，管理程序生命周期和系统交互。
子模块组成：

- 窗口初始化子模块：负责 Windows 窗口类注册和窗口创建
- 消息处理子模块：处理 Windows 消息，协调各父模块工作
- 消息循环子模块：维护程序主事件循环，保持程序运行

协调作用：统一管理图形类父模块和渲染父模块的协作关系。

5.2.5 模块间协作关系

协作流程：

1. 初始化阶段：主程序父模块创建窗口，初始化图形类和渲染父模块
2. 对象创建：图形类父模块按照绘制顺序创建 24 个具体图形对象，对应图形的不同部分：

   图形构成分析：

   (a) 第 **1** 个对象：绘制图形底部的主圆形，作为整个图形的核心基础结构
   (b) 第 **2-3** 个对象：构成图形左侧的大弧形轮廓，形成主体的外围包络线
   (c) 第 **4** 个对象：绘制连接上下部分的中央垂直线段
   (d) 第 **5-9** 个对象：构建图形中部的椭圆形环状结构，包括：
      - 外层椭圆弧线
      - 内层椭圆弧线
      - 左右两条垂直连接线
      - 底部连接弧线
   (e) 第 **10-18** 个对象：构成图形右侧的复杂弧形分支结构，包括：
      - 右侧延伸的连接线段
      - 多个不同半径的同心圆弧
      - 形成右下角的弯曲造型
   (f) 第 **19-23** 个对象：绘制图形顶部的装饰性小圆弧群，包括：
      - 顶端的小椭圆
      - 左右对称的大弧形结构
      - 两个小装饰圆弧

(g) 第 **24** 个对象：绘制从上部装饰结构到右侧分支的斜向连接线

绘制顺序设计原则：

- 从主体到细节：先绘制底部主圆形，再添加各部分细节
- 从内到外：先构建核心结构，再扩展到外围元素
- 分区域绘制：按照底部 → 中部 → 右侧 → 顶部的空间顺序组织
- 最后连接：用连接线将各个独立部分有机结合

3. 渲染阶段：渲染父模块调度图形对象进行批量绘制
4. 交互响应：主程序父模块处理窗口消息，触发重绘和资源调整
5. 清理阶段：按依赖关系逆序释放所有模块资源

5.3 模块化代码详细讲解

本程序采用层次化模块设计，使用 Direct2D API 实现复杂图形的绘制。程序总共 331 行代码，按照父模块-子模块的层次结构进行组织。下面按照模块层次对程序进行详细分析：

模块层次结构说明：

- 父模块：定义主要功能领域，包含多个相关子模块
- 子模块：实现父模块中的具体功能，相互协作完成任务
- 模块间依赖：通过接口和数据传递实现模块间协作

5.3.1 系统依赖模块 - 头文件包含与库链接

```
#include <windows.h>     // Windows API基础头文件
#include <d2d1.h>        // Direct2D图形库头文件
#include <vector>        // STL容器，用于存储图形对象
#include <new>           // 内存分配异常处理
#include <cmath>         // 数学函数库，用于三角函数计算

#pragma comment(lib, "d2d1.lib")  // 链接Direct2D库
```

Listing 1: 系统依赖声明

系统依赖模块负责引入程序所需的所有外部依赖。各头文件的作用如下：

- **windows.h**：提供 Windows API 基础功能，包括窗口创建、消息处理等

- **d2d1.h**：Direct2D 图形库的核心头文件，提供所有绘图接口
- **vector**：STL 容器，用于动态管理图形对象集合
- **new**：提供内存分配异常处理机制
- **cmath**：数学函数库，圆弧绘制需要三角函数计算

5.3.2 父模块：图形类模块（Shapes）

图形类父模块是程序的核心业务模块，负责定义和管理所有图形对象。该父模块包含两个子模块：形状接口子模块和具体形状实现子模块。

```
// Global Direct2D Factory - 图形类父模块的全局资源
ID2D1Factory* g_pFactory = NULL;
```

Listing 2: 图形类父模块全局资源

父模块职责：

- 统一管理 Direct2D 工厂资源，为子模块提供基础设施
- 定义图形系统的整体架构和接口规范
- 协调子模块间的协作关系

子模块 **A**：形状接口子模块

```
// Shape base class - 图形基类
class Shape {
public:
    virtual ~Shape() {}  // 虚析构函数，确保正确释放派生类对象
    // 纯虚函数，定义绘制接口，实现多态
    virtual void Draw(ID2D1RenderTarget* pRT, ID2D1SolidColorBrush*
        pBrush) = 0;
};
```

Listing 3: 形状基类定义

形状接口子模块作为图形类父模块的第一个子模块，定义了所有图形对象的统一接口规范。

子模块 **A** 职责：

- 接口定义：使用纯虚函数 Draw() 定义统一的绘制接口
- 内存管理：虚析构函数确保派生类对象能够正确析构

- 多态支持：为子模块 B 提供策略模式基础
- 参数标准化：统一接口参数：渲染目标 (pRT) 和画刷 (pBrush)

与父模块的关系：

- 依赖父模块提供的 g_pFactory 资源
- 为父模块定义统一的图形操作接口
- 为子模块 B 提供继承基础

子模块 **B**：具体形状实现子模块

具体形状实现子模块作为图形类父模块的第二个子模块，负责实现子模块 A 定义的 Shape 接口。该子模块包含四种具体形状类：直线、矩形、椭圆和圆弧。

子模块 **B** 职责：

- 接口实现：继承子模块 A 的 Shape 基类，实现 Draw() 方法
- 属性封装：封装每种形状的几何属性（坐标、尺寸等）和视觉属性（颜色、线宽）
- 绘制逻辑：将抽象的图形描述转换为 Direct2D API 调用
- 算法实现：对于复杂形状（如圆弧），实现数学计算算法

与父模块和子模块 **A** 的关系：

- 依赖父模块的 g_pFactory 资源进行 Direct2D 操作
- 继承子模块 A 的 Shape 基类，遵循统一接口规范
- 为父模块提供具体的图形绘制能力

1. 直线类（**MyLine**）实现

```
class MyLine : public Shape {
public:
    // 构造函数: 接收起点、终点坐标, 颜色RGB值和线宽
    MyLine(float x1, float y1, float x2, float y2, float r, float g,
       float b, float width) {
        this->x1 = x1; this->y1 = y1; this->x2 = x2; this->y2 = y2;
        this->r = r; this->g = g; this->b = b; this->width = width;
    }

    // 重写基类的Draw方法
    void Draw(ID2D1RenderTarget* pRT, ID2D1SolidColorBrush* pBrush) {
```

```
        pBrush->SetColor(D2D1::ColorF(r, g, b));  // 设置画刷颜色
        // 调用Direct2D API绘制直线
        pRT->DrawLine(D2D1::Point2F(x1, y1), D2D1::Point2F(x2, y2),
            pBrush, width);
    }

private:
    float x1, y1, x2, y2;  // 起点和终点坐标
    float r, g, b;         // RGB颜色值
    float width;           // 线宽
};
```

Listing 4: 直线类实现

MyLine 类设计特点：

- 封装直线的几何属性：起点坐标 (x1,y1)、终点坐标 (x2,y2)
- 封装视觉属性：RGB 颜色值和线宽
- 使用 Direct2D 的 DrawLine API 实现绘制
- 通过 Point2F 结构体传递坐标参数

2. 矩形类（**MyRect**）实现

```
class MyRect : public Shape {
public:
    MyRect(float x1, float y1, float x2, float y2, float r, float g,
        float b, float width) {
        this->x1 = x1; this->y1 = y1; this->x2 = x2; this->y2 = y2;
        this->r = r; this->g = g; this->b = b; this->width = width;
    }

    void Draw(ID2D1RenderTarget* pRT, ID2D1SolidColorBrush* pBrush) {
        pBrush->SetColor(D2D1::ColorF(r, g, b));
        // 使用RectF结构体定义矩形区域
        pRT->DrawRectangle(D2D1::RectF(x1, y1, x2, y2), pBrush, width
            );
    }

private:
    float x1, y1, x2, y2;  // 矩形的左上角和右下角坐标
```

```
    float r, g, b;          // RGB颜色值
    float width;            // 边框线宽
};
```

Listing 5: 矩形类实现

MyRect 类设计特点：

- 使用对角点坐标定义矩形：左上角 (x1,y1) 和右下角 (x2,y2)
- 使用 Direct2D 的 DrawRectangle API 实现绘制
- 通过 RectF 结构体传递矩形区域参数
- 支持边框线宽设置

3. 椭圆类（**MyEllipse**）实现

```
class MyEllipse : public Shape {
public:
    // cx, cy: 椭圆中心坐标; rx, ry: 水平和垂直半径
    MyEllipse(float cx, float cy, float rx, float ry, float r, float
       g, float b, float width) {
        this->cx = cx; this->cy = cy; this->rx = rx; this->ry = ry;
        this->r = r; this->g = g; this->b = b; this->width = width;
    }

    void Draw(ID2D1RenderTarget* pRT, ID2D1SolidColorBrush* pBrush) {
        pBrush->SetColor(D2D1::ColorF(r, g, b));
        // 使用Ellipse结构体定义椭圆
        pRT->DrawEllipse(D2D1::Ellipse(D2D1::Point2F(cx, cy), rx, ry)
           , pBrush, width);
    }

private:
    float cx, cy, rx, ry;  // 中心坐标和半径
    float r, g, b;          // RGB颜色值
    float width;            // 边框线宽
};
```

Listing 6: 椭圆类实现

MyEllipse 类设计特点：

- 使用中心点和半径定义椭圆：中心 (cx,cy)、水平半径 rx、垂直半径 ry

- 使用 Direct2D 的 DrawEllipse API 实现绘制
- 通过 Ellipse 结构体传递椭圆参数
- 支持圆形（rx=ry）和椭圆（rx ry）两种情况

4. 圆弧类（**MyArc**）实现 - 数学计算的应用

圆弧类是具体形状实现子模块中最复杂的部分，因为 Direct2D 没有直接的圆弧绘制 API，需要通过 cmath 库的数学计算实现：

```
class MyArc : public Shape {
public:
    // cx, cy: 圆心坐标; radius: 半径
    // startX, startY: 起始点坐标; endX, endY: 结束点坐标
    // clockwise: 是否顺时针绘制
    MyArc(float cx, float cy, float radius, float startX, float
       startY,
          float endX, float endY, bool clockwise, float r, float g,
             float b, float width) {
        this->cx = cx; this->cy = cy; this->radius = radius;
        this->startX = startX; this->startY = startY;
        this->endX = endX; this->endY = endY;
        this->clockwise = clockwise;
        this->r = r; this->g = g; this->b = b; this->width = width;
    }

private:
    float cx, cy, radius;           // 圆心坐标和半径
    float startX, startY, endX, endY; // 起始点和结束点坐标
    bool clockwise;                 // 绘制方向
    float r, g, b;                  // RGB颜色值
    float width;                    // 线宽
};
```

Listing 7: 圆弧类构造函数和成员变量

MyArc 类设计特点：

- 使用圆心、半径和起始终止点定义圆弧
- 支持顺时针和逆时针两种绘制方向
- 通过数学计算将圆弧分解为多个小线段

- 依赖 cmath 库的三角函数进行角度计算

```
void Draw(ID2D1RenderTarget* pRT, ID2D1SolidColorBrush* pBrush) {
    if (!pRT || !pBrush) return;  // 安全检查

    pBrush->SetColor(D2D1::ColorF(r, g, b));

    const float PI = 3.14159265f;
    // 计算起始角度和结束角度
    float startAngle = (float)atan2(startY - cy, startX - cx);
    float endAngle = (float)atan2(endY - cy, endX - cx);

    // 角度标准化：将负角度转换为正角度
    if (startAngle < 0) startAngle += 2.0f * PI;
    if (endAngle < 0) endAngle += 2.0f * PI;

    // 处理跨越2 边界的情况
    if (!clockwise && endAngle < startAngle) {
        endAngle += 2.0f * PI;
    } else if (clockwise && startAngle < endAngle) {
        startAngle += 2.0f * PI;
    }

    // 将圆弧分解为100个小线段
    int segments = 100;
    float angleStep = 0.0f;

    if (clockwise) {
        angleStep = (startAngle - endAngle) / (float)segments;
    } else {
        angleStep = (endAngle - startAngle) / (float)segments;
    }

    // 绘制每个小线段
    for (int i = 0; i < segments; i++) {
        float angle1 = 0.0f;
        float angle2 = 0.0f;

        if (clockwise) {
            angle1 = startAngle - (float)i * angleStep;
```

```
            angle2 = startAngle - (float)(i + 1) * angleStep;
        } else {
            angle1 = startAngle + (float)i * angleStep;
            angle2 = startAngle + (float)(i + 1) * angleStep;
        }

        // 计算线段端点坐标
        float x1 = cx + radius * (float)cos(angle1);
        float y1 = cy + radius * (float)sin(angle1);
        float x2 = cx + radius * (float)cos(angle2);
        float y2 = cy + radius * (float)sin(angle2);

        // 绘制小线段
        pRT->DrawLine(D2D1::Point2F(x1, y1), D2D1::Point2F(x2, y2),
            pBrush, width);
    }
}
```

Listing 8: 圆弧绘制算法核心实现

圆弧算法关键技术点：

- 使用 atan2 函数计算角度，避免除零错误
- 角度标准化处理，确保角度在 $[0, 2\pi]$ 范围内
- 正确处理顺时针和逆时针方向
- 将圆弧分解为 100 个小线段，保证绘制平滑度
- 使用三角函数 cos 和 sin 计算圆弧上的点坐标

5.3.3 父模块：渲染模块（Renderer）

渲染父模块负责管理整个绘制流程，包含两个子模块：D2D 资源管理子模块和绘制调度子模块。该父模块与图形类父模块紧密协作，实现图形的实际渲染。

```
// Global shape list - 渲染父模块的全局资源
std::vector<Shape*> g_shapes;
```

Listing 9: 渲染父模块全局资源

父模块职责：

- 统一管理所有待绘制的图形对象

- 协调 D2D 资源管理和绘制调度两个子模块
- 与图形类父模块协作，实现图形的渲染输出

子模块 **A**：绘制调度子模块
绘制调度子模块设计优点：

- 动态管理：支持运行时动态添加、删除图形对象
- 多态统一：利用 Shape 基类指针，统一处理不同类型的图形
- 高效遍历：支持快速遍历和批量绘制操作
- 解耦设计：不直接操作 D2D 资源，依赖 D2D 资源管理子模块

绘制调度流程：

1. 初始化阶段：在 WM_CREATE 消息中创建所有图形对象
2. 绘制阶段：在 WM_PAINT 消息中遍历 g_shapes 容器
3. 清理阶段：在 WM_DESTROY 消息中释放所有图形对象

5.3.4 父模块：主程序模块（Main）

主程序父模块是整个程序的最高层控制模块，负责协调所有其他父模块的工作。该父模块包含三个子模块：窗口初始化子模块、消息处理子模块和消息循环子模块。
父模块职责：

- 统一管理程序的生命周期：初始化、运行、退出
- 协调图形类父模块和渲染父模块的工作
- 处理系统级事件和用户交互

子模块 **A**：消息处理子模块
消息处理子模块是主程序父模块的核心子模块，负责响应 Windows 消息并协调其他父模块的工作。主要通过 WndProc 窗口过程函数实现。
子模块 **A** 职责：

- 消息分发：接收并分类处理各种 Windows 消息
- 模块协调：根据消息类型调用相应的父模块功能
- 生命周期管理：管理程序的初始化、运行和清理阶段

与其他父模块的协作关系：

- **WM_CREATE**：初始化图形类父模块和渲染父模块
- **WM_PAINT**：协调渲染父模块绘制图形类父模块的对象
- **WM_DESTROY**：清理所有父模块的资源

1. **WM_CREATE** 消息处理 - 系统初始化

```
case WM_CREATE: {
    // 创建Direct2D工厂 - D2D资源管理子模块初始化
    HRESULT hr = D2D1CreateFactory(D2D1_FACTORY_TYPE_SINGLE_THREADED,
        &g_pFactory);
    if (FAILED(hr) || !g_pFactory) {
        MessageBox(hwnd, "Failed to create Direct2D factory", "Error"
            , MB_OK | MB_ICONERROR);
        return -1;
    }

    // 绘制调度子模块初始化 - 创建图形对象集合
    try {
        // 按照实际图形需求创建各种形状对象
        g_shapes.push_back(new MyEllipse(150, 300, 40, 40, 0.0f, 0.0f
            , 0.0f, 2.0f));
        g_shapes.push_back(new MyArc(150, 300, 64, 212.98, 311.38,
            109.28, 250.62, false, 0.0f, 0.0f, 0.0f, 2.0f));
        g_shapes.push_back(new MyArc(87, 223.6, 35, 109.28, 250.62,
            122, 223.6, true, 0.0f, 0.0f, 0.0f, 2.0f));
        g_shapes.push_back(new MyLine(122, 134, 122, 223.6, 0.0f, 0.0
            f, 0.0f, 2.0f));
        // ... 更多图形对象创建（共创建24个图形对象）
    }
    catch (const std::bad_alloc&) {
        MessageBox(hwnd, "Memory allocation failed", "Error", MB_OK |
             MB_ICONERROR);
        return -1;
    }
    return 0;
}
```

Listing 10: 系统初始化消息处理

WM_CREATE 消息处理关键技术点：

- 资源创建检查：使用 HRESULT 检查 Direct2D 工厂创建是否成功
- 异常安全：采用 try-catch 机制保护内存分配操作
- 批量初始化：一次性创建所有需要的图形对象（本程序创建 24 个）
- 错误处理：创建失败时显示错误信息并退出

2. WM_PAINT 消息处理 - 绘制流程协调

```
case WM_PAINT: {
    PAINTSTRUCT ps;
    BeginPaint(hwnd, &ps);  // 开始Windows绘制

    // D2D资源管理子模块 - 按需创建渲染目标
    if (!pRT && g_pFactory) {
        RECT rc;
        GetClientRect(hwnd, &rc);  // 获取窗口客户区尺寸

        if (rc.right > 0 && rc.bottom > 0) {
            // 创建硬件加速的窗口渲染目标
            HRESULT hr = g_pFactory->CreateHwndRenderTarget(
                D2D1::RenderTargetProperties(),
                D2D1::HwndRenderTargetProperties(hwnd, D2D1::SizeU(rc
                    .right, rc.bottom)),
                &pRT
            );

            // 创建实心画刷资源
            if (SUCCEEDED(hr) && pRT) {
                hr = pRT->CreateSolidColorBrush(D2D1::ColorF(0, 0, 0)
                    , &pBrush);
            }

            // 资源创建失败处理
            if (FAILED(hr)) {
                if (pBrush) { pBrush->Release(); pBrush = NULL; }
                if (pRT) { pRT->Release(); pRT = NULL; }
                EndPaint(hwnd, &ps);
                return 0;
            }
        }
```

```
31      }
32
33      // 绘制调度子模块 - 执行实际绘制操作
34      if (pRT && pBrush) {
35          pRT->BeginDraw();  // 开始Direct2D绘制会话
36          pRT->Clear(D2D1::ColorF(1.0f, 1.0f, 1.0f));  // 清空背景为白
                色
37
38          // 遍历绘制调度列表，批量绘制所有图形
39          for (size_t i = 0; i < g_shapes.size(); ++i) {
40              if (g_shapes[i]) {
41                  g_shapes[i]->Draw(pRT, pBrush);  // 多态调用各形状的
                        Draw方法
42              }
43          }
44
45          HRESULT hr = pRT->EndDraw();  // 结束Direct2D绘制会话
46          // 处理设备丢失情况（如全屏切换等）
47          if (hr == D2DERR_RECREATE_TARGET) {
48              if (pBrush) { pBrush->Release(); pBrush = NULL; }
49              if (pRT) { pRT->Release(); pRT = NULL; }
50          }
51      }
52
53      EndPaint(hwnd, &ps);  // 结束Windows绘制会话
54      return 0;
55  }
```

Listing 11: 绘制流程消息处理

WM_PAINT 消息处理关键技术：

- 延迟创建：渲染目标只在需要时创建，节省资源
- 硬件加速：使用 HwndRenderTarget 利用 GPU 加速
- 设备管理：正确处理设备丢失（D2DERR_RECREATE_TARGET）情况
- 多态绘制：利用多态机制统一处理不同类型图形
- 标准流程：清空背景 → 遍历形状列表 → 批量绘制

3. WM_SIZE 消息处理 - 窗口尺寸自适应

```
case WM_SIZE: {
    // D2D资源管理子模块 - 动态调整渲染目标尺寸
    if (pRT) {
        RECT rc;
        GetClientRect(hwnd, &rc);  // 获取新的客户区尺寸
        if (rc.right > 0 && rc.bottom > 0) {
            // 调整渲染目标尺寸以匹配新窗口大小
            HRESULT hr = pRT->Resize(D2D1::SizeU(rc.right, rc.bottom)
                );
            if (FAILED(hr)) {
                // 尺寸调整失败，释放资源以便下次重新创建
                if (pBrush) { pBrush->Release(); pBrush = NULL; }
                if (pRT) { pRT->Release(); pRT = NULL; }
            }
            InvalidateRect(hwnd, NULL, FALSE);  // 触发窗口重绘
        }
    }
    return 0;
}
```

Listing 12: 窗口尺寸变化处理

WM_SIZE 消息处理关键技术：

- 动态调整：渲染目标尺寸随窗口大小变化而调整
- 失败处理：调整失败时释放资源，下次绘制时重新创建
- 重绘触发：使用 InvalidateRect 触发窗口重绘

4. WM_DESTROY 消息处理 - 系统资源清理

```
case WM_DESTROY:
    // 绘制调度子模块清理 - 释放所有图形对象
    for (size_t i = 0; i < g_shapes.size(); ++i) {
        if (g_shapes[i]) {
            delete g_shapes[i];  // 释放图形对象内存
            g_shapes[i] = NULL;  // 避免悬挂指针
        }
    }
    g_shapes.clear();  // 清空容器

```

```
    // D2D资源管理子模块清理 - 按相反顺序释放资源
    if (pBrush) { pBrush->Release(); pBrush = NULL; }
    if (pRT) { pRT->Release(); pRT = NULL; }
    if (g_pFactory) { g_pFactory->Release(); g_pFactory = NULL; }

    PostQuitMessage(0);  // 发送程序退出消息
    return 0;
```

Listing 13: 系统资源清理处理

WM_DESTROY 消息处理关键技术：

- 内存清理：逐个释放所有图形对象，防止内存泄漏
- 指针安全：释放后将指针设为 NULL，避免悬挂指针
- 资源顺序：按创建的相反顺序释放资源，确保依赖关系正确
- 程序退出：使用 PostQuitMessage 发送退出消息

5.3.5 主程序模块（Main）- 窗口初始化子模块和消息循环子模块

窗口初始化子模块负责 Windows 窗口系统的初始化工作，包括窗口类注册和窗口创建。消息循环子模块负责程序的主事件循环，持续接收、处理和分发 Windows 消息。

```
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
    LPSTR lpCmdLine, int nCmdShow) {
    const char CLASS_NAME[] = "Direct2D Window";

    // 窗口初始化子模块 - 注册窗口类
    WNDCLASS wc = { 0 };
    wc.lpfnWndProc = WndProc;           // 设置消息处理函数
    wc.hInstance = hInstance;           // 应用程序实例句柄
    wc.lpszClassName = CLASS_NAME;      // 窗口类名标识符
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);  // 默认背景画刷

    RegisterClass(&wc);  // 向系统注册窗口类

    // 窗口初始化子模块 - 创建窗口实例
    HWND hwnd = CreateWindowEx(
        0,                              // 扩展窗口样式
        CLASS_NAME,                     // 使用已注册的窗口类
        "Direct2D Multi-Shape Drawing", // 窗口标题栏文本
```

```
        WS_OVERLAPPEDWINDOW,            // 标准重叠窗口样式
        CW_USEDEFAULT, CW_USEDEFAULT,   // 系统默认位置
        400, 500,                       // 窗口初始尺寸
        NULL, NULL, hInstance, NULL     // 父窗口、菜单、实例、参数
    );

    if (!hwnd) return 0;  // 窗口创建失败则退出程序

    ShowWindow(hwnd, nCmdShow);  // 显示窗口并设置显示状态

    // 消息循环子模块 - 主事件循环
    MSG msg = { 0 };
    while (GetMessage(&msg, NULL, 0, 0)) {  // 获取消息，阻塞等待
        TranslateMessage(&msg);  // 翻译虚拟键消息
        DispatchMessage(&msg);   // 分发消息到窗口过程函数
    }

    return 0;  // 程序正常退出
}
```

Listing 14: 主程序入口函数实现

主程序模块关键技术点：

- 窗口类注册：定义窗口的基本属性和行为，包括消息处理函数
- 窗口创建：使用 CreateWindowEx 创建具体窗口实例
- 消息循环：使用 GetMessage-TranslateMessage-DispatchMessage 模式
- 错误处理：检查窗口创建是否成功，失败时退出

父子模块协作流程总结：

1. 程序启动：主程序父模块的窗口初始化子模块创建窗口载体
2. 资源初始化：消息处理子模块接收 WM_CREATE，初始化图形类父模块和渲染父模块
3. 图形创建：图形类父模块的具体形状实现子模块创建 24 个图形对象
4. 绘制执行：消息处理子模块接收 WM_PAINT，协调渲染父模块的两个子模块完成绘制

5. 尺寸调整：消息处理子模块接收 WM_SIZE，调用渲染父模块的 D2D 资源管理子模块
6. 资源清理：消息处理子模块接收 WM_DESTROY，清理所有父模块的资源
7. 消息循环：主程序父模块的消息循环子模块维持程序运行

模块间依赖关系图：

- 主程序父模块 → 协调 → 图形类父模块 + 渲染父模块
- 图形类父模块：形状接口子模块 → 定义接口 → 具体形状实现子模块
- 渲染父模块：绘制调度子模块　协作　D2D 资源管理子模块
- 主程序父模块：窗口初始化 → 消息处理 → 消息循环

5.4 程序运行结果

程序成功编译运行后，能够绘制出要求的复杂图形。

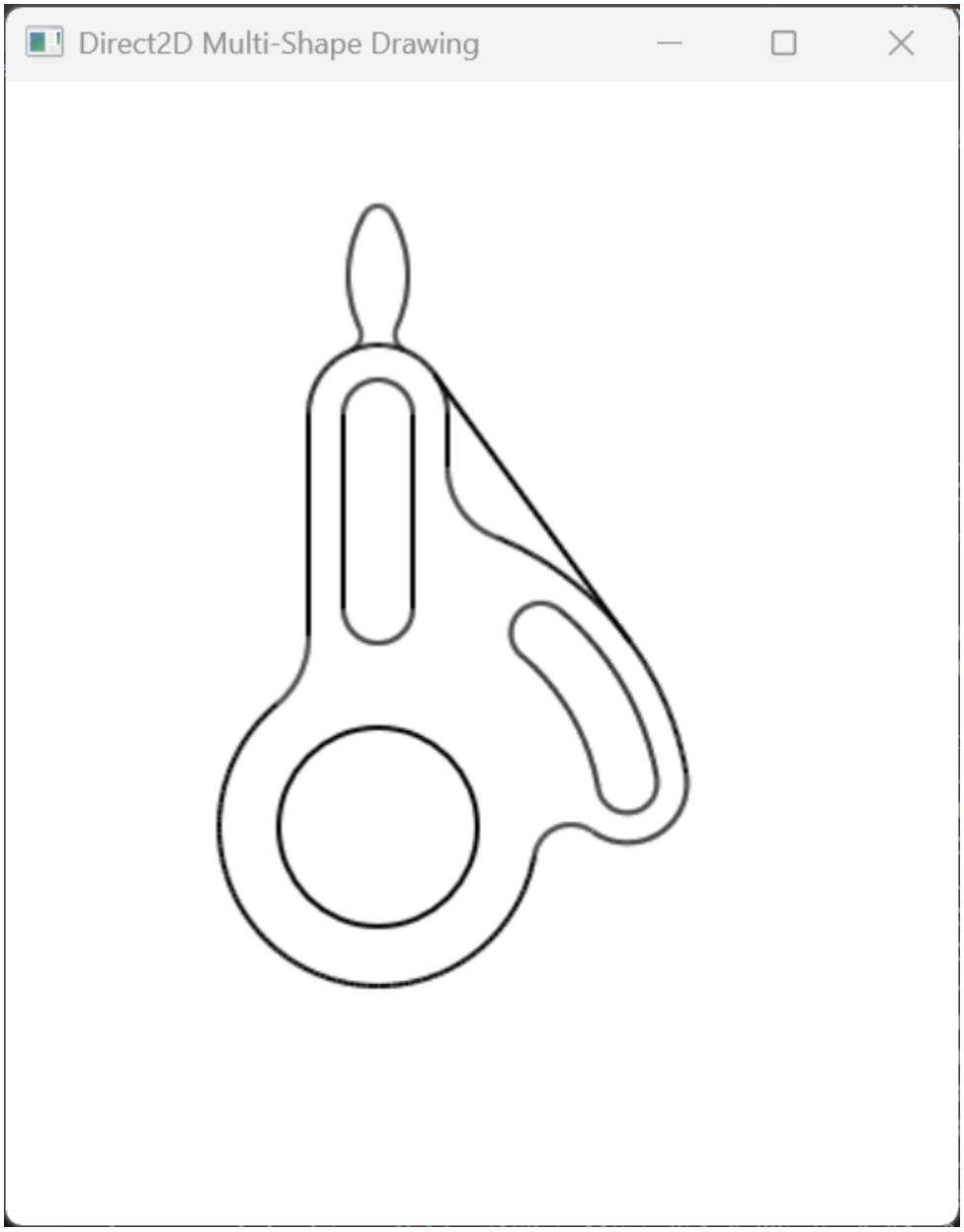


图 10: 程序运行结果截图

6 实验总结

通过本次实验，我深入学习了 Direct2D 图形编程技术，主要收获如下：

1. 技术掌握：熟练掌握了 Direct2D API 的使用方法，包括工厂创建、渲染目标管理、图形绘制等核心技术。
2. 设计思想：学会了面向对象的图形程序设计方法，通过继承和多态实现了灵活的图形绘制框架。

3. 数学应用：将数学知识（三角函数、解析几何）应用到实际编程中，解决了复杂圆弧绘制问题。

4. 资源管理：深入理解了 Windows 图形资源的生命周期管理，学会了正确的创建和释放策略。

5. 调试技能：通过解决各种编程问题，提高了程序调试和错误处理能力。

本实验为后续的计算机图形学学习奠定了坚实的基础，不仅掌握了基本的图形编程技术，更重要的是培养了系统性的程序设计思维和解决复杂问题的能力。